

Reviewer Pack — Minimal Order of Groupoid Representations and Communication ...

Entry 9c5d9bbb962f · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 10:23:13 UTC

Minimal Order of Groupoid Representations and Communication Complexity Rank Inequality

Entry ID: 9c5d9bbb962f **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-05 10:23:13 UTC

1. Conjecture

Field A (mathematical branch): Category Theory (Groupoids) **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

For every Boolean function f with m inputs, the minimal order of groupoid representations that can distinguish f from its complement is $\Omega(m^2 \cdot \log m)$, and this lower bound is tight for specific functions.

Rationale (proposer's reasoning):

Groupoids provide a noncommutative generalization of groups, which may offer a new perspective on communication complexity. The use of groupoid representations to encode Boolean functions could lead to new insights into the structure of communication complexity problems.

Taxonomy category: Category Theory (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 54d8d25e1bb1945b

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

If for any Boolean function f with m inputs, the computed minimal order of groupoid representations distinguishing f from its complement is greater than or equal to $\Omega(m^2 \cdot \log m)$ and at least 80% of seeds show this inequality, then the conjecture is supported.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "groupoid representation" AND "communication complexity" AND "matrix rank" - "Category Theory" AND "Boolean function" AND " $\Omega(m^2 * \log m)$ " - "minimal order groupoid representations" AND "lower bound communication complexity"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.5s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(m):
        return [random.choice([0, 1]) for _ in range(2**m)]

    def communication_complexity_rank(f):
        n = len(f)
        if n <= 1:
            return 0
        rank = 0
        for i in range(n):
            if f[i] != f[~i]:
                rank += 1
        return rank

    m_values = [5, 10, 15, 20, 30, 40]
    results = []

    for m in m_values:
        f = generate_boolean_function(m)
        rank = communication_complexity_rank(f)
        results.append({
            "m": m,
            "rank": rank
        })

    metric_value = sum(result["rank"] for result in results) / len(results)
    instances_tested = len(results)
```

```

n_max = max(result["m"] for result in results)
conjecture_holds = all(rank >= 2 * m**2 * math.log(m) for result in results)

return {
    "metric_name": "Communication Complexity Rank",
    "metric_value": metric_value,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    std_value = math.sqrt(sum((result["metric_value"] - mean_value)**2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='not supported' first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means it did not provide evidence to support or falsify the conjecture. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13806
2	propose	ollama_remote	glm4:latest	0	0	13328
3	preregistration	ollama_remote	glm4:latest	0	0	9353
4	novelty	ollama_remote	glm4:latest	0	0	8514
5	novelty	ollama_remote	glm4:latest	0	0	9155
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12703
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8127
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8255
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7525
10	judge	ollama_remote	glm4:latest	0	0	14915

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 105680 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/9c5d9bbb962f.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/9c5d9bbb962f.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/9c5d9bbb962f.tar.gz (if generated)